

Dhyana Stays Tech Stack

A snapshot of the frameworks, libraries and tooling that power the product today — pulled directly from `package.json`, config files and the app's own architecture.

Next.js 16

React 19

TypeScript 5

Tailwind CSS v4

Framer Motion

Turbopack

Architecture at a Glance

A single Next.js application. No separate backend service exists yet — content is served from a typed mock-data layer, and user actions (wishlist, itinerary, prebook) persist client-side.

Presentation
React 19 Server & Client Components, App Router pages, shared UI components

NEXT.JS 16

Styling
Tailwind CSS v4 utility classes + a custom CSS-variable design-token system (globals.css)

TAILWIND V4

Interaction & Motion
Framer Motion for transitions/gestures, next-themes for light/dark, Lucide for icons

CLIENT LAYER

State & Data
React Context (theme, location, search) + custom hooks backed by typed mock-data modules and localStorage — no database or API yet

MOCK DATA

Tooling
TypeScript strict mode, ESLint 9, Turbopack (dev/build), Playwright for browser automation

DX

LAYER	DECISION	WHY IT MATTERS
Rendering	Next.js App Router, route groups (no API routes)	Clean URL structure across public/dashboard/booking flows without exposing folder grouping
Persistence	localStorage only (e.g. wishlist, itinerary)	Real interactive behavior for the prototype without fabricating a backend
Theming	CSS custom properties + .dark class	One token set drives both light and dark mode consistently

Core Framework & Runtime

The foundation every page is built on.

Next.js

16.2.10

FRAMEWORK

App Router, file-based route groups (`(public)` , `(dashboard)` , `(booking)`), `next/font` for self-hosted Google Fonts, Turbopack as the default dev/build engine.

React

19.2.4

UI LIBRARY

Server + Client Components split via `"use client"` directives; hooks-driven state throughout dashboards and interactive widgets.

TypeScript

^5

LANGUAGE

Strict mode enabled, path alias `@/*` → project root, typed mock-data modules act as the app's implicit schema.

React DOM

19.2.4

RENDERING

Paired 1:1 with the React version per Next.js 16 requirements.

BUILD ENGINE

COMMAND	ENGINE	NOTES
<code>next dev</code>	Turbopack	Default dev server — fast refresh, incremental compilation
<code>next dev --webpack</code>	Webpack	Fallback dev script kept for parity/debugging
<code>next build</code> / <code>next start</code>	Turbopack	Production build + Node server

Styling, UI & Motion

No component library (no shadcn/MUI/Chakra) — every button, card and pill is a hand-built component styled directly with Tailwind + design tokens.

Tailwind CSS

^4

STYLING

v4's CSS-first config — theme lives in `app/globals.css` via `@theme inline` rather than a `tailwind.config.js`.

@tailwindcss/postcss

^4

BUILD

PostCSS plugin wiring Tailwind v4 into the Next.js build pipeline.

Framer Motion

^12.42.2

ANIMATION

Page/element transitions, gesture-driven interactions across dashboard and trip-planner widgets.

Lucide React

^1.24.0

ICONS

Single icon set used app-wide — buttons, nav, cards, dashboard tiles.

next-themes

^0.4.6

THEMING

Drives the `.dark` class toggle; wrapped by the app's own `ThemeProvider`, defaults to light with system detection.

next/font

built-in

TYPOGRAPHY

Self-hosts Inter (sans/body) and Playfair Display (serif/display headings) — zero layout shift, no external font requests.

Data, State & Routing

No external state library (no Redux/Zustand/React Query) — plain React Context + custom hooks are enough for a mock-data-driven prototype.

CONCERN	MECHANISM	WHERE
Theme (light/dark)	React Context wrapping <code>next-themes</code>	<code>app/components/ThemeProvider.tsx</code>
User location	React Context + geolocation hook	<code>lib/UserLocationContext</code> , <code>lib/useUserLocation.ts</code>
Search query (global)	React Context	<code>lib/useSearchQuery</code>
Wishlist	Custom hook + <code>localStorage</code>	<code>lib/useWishlist.ts</code>
Trip itinerary	Custom hook + <code>localStorage</code>	<code>lib/useItinerary.ts</code>
Pre-book requests	Custom hook + <code>localStorage</code>	<code>lib/usePrebookRequests.ts</code>
Content (properties, experiences, guides, dashboards...)	Typed static modules — no fetch, no DB	<code>lib/mock-data*.ts</code> , <code>lib/dashboards.ts</code> , <code>lib/trip- dashboard-data.ts</code>

APP ROUTER — ROUTE GROUPS

(public)

- explore-stays, stays
- experiences, destinations
- event-planners, bike-rental
- vaksana-farms, blog, faq
- business, careers, contact

(dashboard)

- [role] — dynamic role shell
- host, super-host, admin
- traveller, investor, finance
- influencer, marketing
- event-planner, sos-manager...

(booking) + auth

- book/[id], payment
- confirmation
- login, register
- forgot-password
- verify-otp

Tooling, Quality & What's Missing

Current developer tooling, plus the honest gaps between "prototype" and "production" — useful for planning the next phase.

ESLint

^9

LINT

Flat config (`eslint.config.mjs`) extending `eslint-config-next` 16.2.10.

Playwright

^1.62.1

AUTOMATION

Browser automation dependency for driving/screenshooting the running app — no committed test suite yet.

NPM SCRIPTS

SCRIPT	COMMAND
<code>npm run dev</code>	<code>next dev</code> (Turbopack)
<code>npm run dev:webpack</code>	<code>next dev --webpack</code>
<code>npm run build</code>	<code>next build</code>
<code>npm start</code>	<code>next start</code>
<code>npm run lint</code>	<code>eslint</code>

NOT YET IN THE STACK



No database

All content is hard-coded in `lib/*` — no Postgres/MySQL/Mongo/ORM present.



No backend / API routes

No `app/api` handlers, no server actions wired to persistence — everything client-rendered from mock data.



No real authentication

Login/register/OTP/forgot-password screens exist as UI only — no session/auth provider (NextAuth, Clerk, etc.) is wired in.



No payment gateway

The payment screen is a UI flow — no Stripe/Razorpay SDK is integrated.



No deployment config committed

No `vercel.json` or CI workflow in the repo yet — Next.js 16 deploys cleanly to Vercel with no config if/when that's set up.

